zorp: A Standalone-Capability Harness for Evidence-Based Investigation

Aviskaar

August 2026

Abstract

Most autonomous-research agents are built around a single assumption: that the deliverable is a finished document an AI produced end to end. That assumption does not survive contact with how research and technical decisions actually get published or adopted, since work with no human author of record is rejected outright at most venues and distrusted in most organizations. We present zorp, a harness that instead decomposes evidence-based investigation into four standalone capabilities – *validate*, *investigate*, *co-write*, and *deliver* – any one of which can be used alone, chained through human-reviewed checkpoints, and built on a shared foundation that keeps a tamper-evident, git-backed record of what was tried. We describe the system's architecture, its data model, and the design decisions that follow from treating the human as the author of record rather than a downstream reviewer. zorp is implemented in Rust as a single multi-subcommand binary (`zorp-agent`) with an optional research feature, is early and pre-alpha, and this paper is a systems and design report rather than a benchmark study: we describe what is built and tested (all four capabilities, 469 passing tests in the default workspace, 445 in `zorp-agent` with the research feature enabled, as of 2026-08-13) and are explicit about what remains open, chiefly a comparative evaluation against systems like AI-Scientist-v2.

Introduction

An uncertain question, worked all the way through to a defensible answer, has the same shape whether it is "should we migrate off Kafka to Redpanda," a competitive teardown, an investment thesis, a due-diligence package, or an academic hypothesis: state the question, gather evidence, reason about conflicting evidence, produce an answer or artifact, and be able to show your work. zorp is a harness for that shape of problem. It turns an uncertain question into evidence, evidence into a defensible answer, and keeps a record of how it got there.

The dominant pattern in "AI scientist" systems – Sakana AI's AI-Scientist-v2 chief among them, and, closer to home, Aviskaar's own internal lab-engine/Catalyst pipeline – wires a large agent framework directly to experiment code and treats "write the paper" as one more autonomous step, with a fact-check or a review pass bolted on at the end. That pattern solves a narrower problem than the one that actually matters: a paper an AI wrote end to end is not submittable at most venues, and a technical recommendation nobody on a team can vouch for is not actionable, regardless of

how sound the underlying reasoning was. Treating authored-document generation as the finish line is solving the wrong problem. zorp starts from the opposite end: the harness is built so that a human is the author of record for whatever gets produced – whether that is a paper, a decision memo, or a competitive landscape – and the system's job is to make that human's evidence trail complete, inspectable, and hard to quietly tamper with after the fact.

This paper describes zorp's design: why it is shaped as four standalone capabilities rather than one pipeline, what shared foundation they sit on, how the checkpoint pattern keeps a human in the loop without making every capability interactive-only, and what is built and tested as of this writing. It is a systems and design paper, not an empirical comparison; we are explicit in Section 7 about what a rigorous comparison against AI-Scientist-v2 and similar systems would require and why we have not yet run one.

Related Work

AI-Scientist-v2 (Sakana AI) automates the full research loop – ideation, experimentation, and paper writing – with the paper as an autonomously produced artifact, subjected to an automated review step after the fact. This is the clearest example of the assumption zorp argues against: treating the finished document as something the system produces, rather than something a human produces with the system's help.

Aviskaar's lab-engine/Catalyst is a working idea-to-paper pipeline built to bootstrap Aviskaar's own sub-projects, with real, useful discipline this paper borrows directly: capped, cheap validation experiments and a `prereg.md` convention – a git-committed, human-readable pre-registration file whose commit timestamp cannot be quietly moved after results are seen. zorp adopts the same tamper-evidence idea for its own pre-registration but does not adopt Catalyst's hard experiment budget (150 lines of code, 10 minutes, no GPU); that cap is well-tuned for Catalyst's narrow validation-experiment use case but would be arbitrary for zorp's broader scope of arbitrary evidence-based questions, so zorp ships guidance rather than enforcement.

Open Research Review (ORR), an Aviskaar-internal system, does real, overlapping work in tracking and experiment state. zorp deliberately does not take a hard dependency on it, or on any other Aviskaar-private infrastructure: zorp has to work for someone who has never heard of Aviskaar, so it owns its own run record (Section 4) rather than requiring a private service most users cannot install.

Across all three systems, the common thread zorp diverges on is where the human sits. AI-Scientist-v2 and, to a lesser extent, Catalyst place the human after the fact, reviewing an output. zorp places the human inside the loop at explicit checkpoints and as the author of record for the two capabilities (co-write, deliver) that produce human-facing artifacts.

System Overview

zorp is a single Rust workspace forked from quecto, a minimal, vendor-neutral harness for LLM agents (MIT licensed), and extended directly rather than depended on as an external crate, since zorp's needs – long-running research loops, experiment tracking, paper synthesis – diverge substantially from a general agent harness. The workspace has five crates: a core transport crate (`src/`, binary `zorp`) with the raw model-calling primitives; `zorp-agent`, the full agent with tools,

sandboxing, trust levels, verification, sessions, and MCP integration, compiled to a single binary; `zorp-mcp`, the Model Context Protocol client/server integration; `zorp-eval`, a deterministic evaluation harness; and `zorp-track`, zorp's own research foundation, described in Section 4.

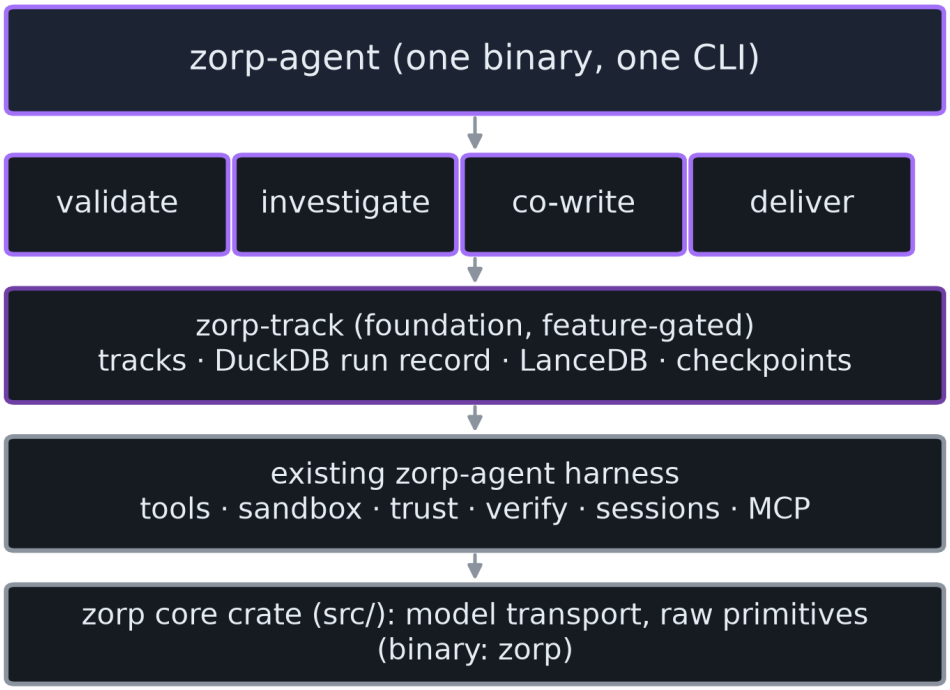

Figure 1: Layered architecture. Each of the four capabilities is a `zorp-agent` subcommand, not a separate program; parallel workers are additional copies of the same binary run as isolated subprocesses.

One binary, not one pipeline

`zorp-agent` gains subcommands for all four capabilities rather than a separate `zorp-research` binary that shells out to it. This is a deliberate simplicity choice: one thing to install, learn, and support, and parallel experiment workers (for `investigate`) are isolated subprocesses spawned as more copies of `zorp-agent` itself, not a second program with its own lifecycle.

Feature-gating the heavy dependencies

`zorp-track` bundles DuckDB (via `duckdb`, bundled) and provisions a LanceDB store, which pulls in Arrow and DataFusion. These are substantial dependencies – compiling them from a cold cache takes 20 to 30 minutes – so `zorp-agent` depends on `zorp-track` behind an optional `research` Cargo feature, the same pattern already used for `zorp-mcp` behind an `mcp` feature and for OpenTelemetry behind an `otel` feature. A user who only wants the base agent (tools, sandbox, verification, MCP, no research capabilities) never pays that compilation cost; `cargo build --workspace --exclude zorp-track` is the documented fast path, and CI itself runs the excluded form for the same reason.

The Shared Foundation: zorp-track

None of the four capabilities can exist without a record of what was tried, a place to put it, and a mechanism for pausing at points where a human needs to decide something. `zorp-track` is that foundation. It owns the track data model, the DuckDB run record, the LanceDB store, the pre-registration file and row management, and the checkpoint primitive. It does not know about validate, investigate, co-write, or deliver – they are built on top of it, not into it, which keeps the foundation testable independent of any specific capability's logic.

Two stores, split by job

zorp splits its data across two stores by what kind of question each answers. **DuckDB** holds the transactional and analytical run record: tracks, pre-registrations, experiments, and typed metrics (a metric is a `(key, value_type, value)` tuple, where `value_type` is one of `number`, `string`, or `bool` and the value lives in whichever typed column matches). This typed-not-narrative choice matters directly for co-write: when co-write drafts a claim like "the p99 latency dropped by 40%," it can check that claim against an exact recorded number, not grep a log file and hope. **LanceDB** is provisioned for multimodal, semantically searchable content – literature embeddings, figures, plots – keyed by track id so later capabilities can scope a search to one track. As of this writing LanceDB is provisioned but has no producers or consumers yet; each capability's own use of it is left to that capability's own design.

Pre-registration as tamper evidence

Every track requires pre-registration before any investigative work begins: a hypothesis, a metric name, and a numeric kill threshold, written to a human-readable `prereg.md` file and committed to git before any experiment code runs. The choice of a git-committed file, rather than only a database row, is deliberate: a git commit timestamp cannot be quietly moved after results are seen, which is exactly the guarantee a database row alone cannot provide. `zorp.duckdb` and the LanceDB store are gitignored, regenerable indexes over these files, not the source of truth – if either is lost or corrupted, `zorp-track` rebuilds the index by re-reading every `prereg.md` under `tracks/`. On every track load, `zorp-track` re-hashes each `prereg.md` (SHA-256 of the raw bytes) and compares it against the hash recorded at commit time; a mismatch, whether a missing file, a missing database row, or content that no longer matches its recorded hash, is a hard error, not a warning. This is what makes the tamper-evidence guarantee real: a threshold changed after the fact, without a new commit, is detected on the next load, not silently accepted.

The checkpoint pattern

A `Checkpoint` type gates progress at track granularity, mirroring the shape of `zorp-agent`'s existing per-tool-call `Approver` trait and `ApprovalMode` enum but applied at a coarser grain. Two modes: `Interactive` (default, blocks synchronously and prompts a human) and `AutoApprove` (explicit opt-in, for unattended runs via the `--yes` flag). Deliberately, there is no silent-skip default: a checkpoint reached in a non-interactive terminal without `AutoApprove` set is a hard error, because unlike a single tool call, a research checkpoint has no safe default to fall back to when nobody is there to answer. Every checkpoint records what was shown to the human (`prompt_shown`) and their decision (`decision_notes`), so a track's history captures not just what was tried, but what a human was asked and how they answered.

The Four Capabilities

Each capability is a standalone `zorp-agent` subcommand. A "full loop" is the four of them chained together with a human checkpoint between each step – not a separate mode, just what happens when someone runs all four in sequence. This framing follows directly from how the harness is actually used: someone validating a single idea, or running one investigation, without wanting to commit to the full loop, is at least as common a case as someone who wants all four, and forcing everything through one pipeline would serve the full-loop case at the expense of the much more common partial one.

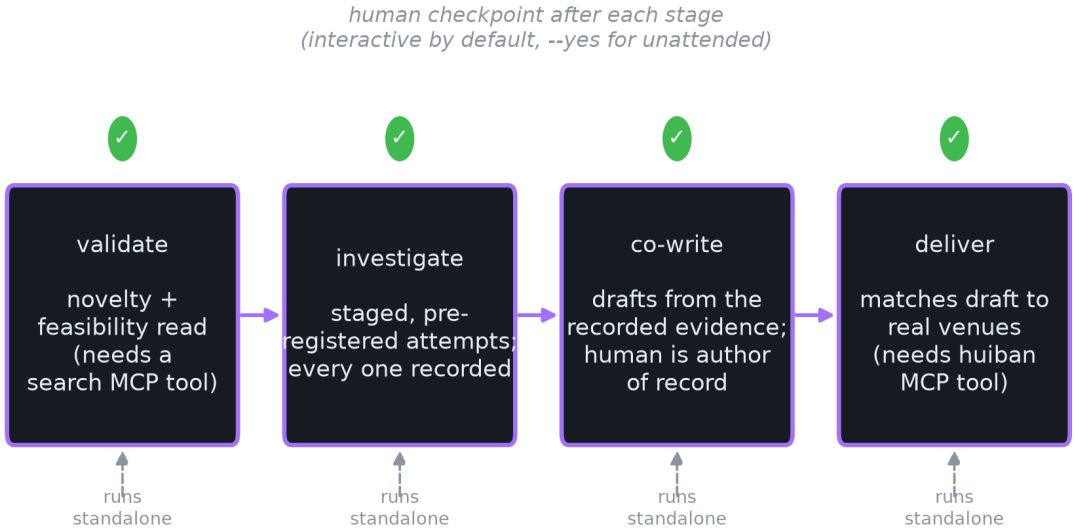

Figure 2: The four capabilities as a checkpointed pipeline. Each stage also runs standalone; validate and deliver additionally require an MCP tool to be configured before they will run.

validate takes a question, searches for evidence using whichever MCP tools are configured, and scores redundancy and feasibility with required citations before checkpointing. It fails fast, before doing any uncited scoring, if no MCP tool with search capability is configured (any `mcp__`-prefixed tool satisfies this gate) – a deliberate choice to refuse running with no evidence rather than produce an unsupported score.

investigate gathers evidence through staged, pre-registered attempts. Each invocation runs one attempt against a CLI-supplied metric name and kill threshold; every attempt is recorded in the run record, not just the best one, and a human checkpoint decides whether to kill the track or keep going after each attempt. Recording every attempt, including the ones that did not pan out, is what makes the record useful as evidence later, rather than a highlight reel.

co-write drafts the artifact directly from the track's recorded evidence: validate's verdict, if one exists, and every metric investigate recorded, handed to the agent as structured data with instructions to cite only those figures. It writes directly to `draft.md`. Critically, a human remains the author of record – co-write drafts, it does not finalize, and rejecting its checkpoint does not kill the track, since a draft that needs another pass is not a failed investigation.

deliver takes the finished `draft.md` and matches it against real venues. Scoped to academic venue-matching for its first version, it requires a huiban-prefixed MCP tool (huiban is a live conference and journal database) to be configured, gated the same way validate's search-tool requirement is gated, and writes a ranked shortlist to `venues.md` for a human to review. Like co-write, rejecting deliver's checkpoint does not kill the track.

Two of the four – validate and deliver – have a hard external dependency on an MCP tool being configured; investigate and co-write do not, since they work entirely from the track's own recorded evidence.

Implementation Status

As of 2026-08-13, all four capabilities are built and tested, sitting on a `zorp-track` foundation that is itself built and tested. Table 1 and Figure 3 report test counts from `cargo test --workspace --exclude zorp-track` (the default workspace, matching what CI runs) and from `cargo test -p zorp-agent --features research` (which exercises validate, investigate, co-write, and deliver, including integration tests that spin up a stub MCP server over stdio to verify the tool-gate and round-trip logic end to end).

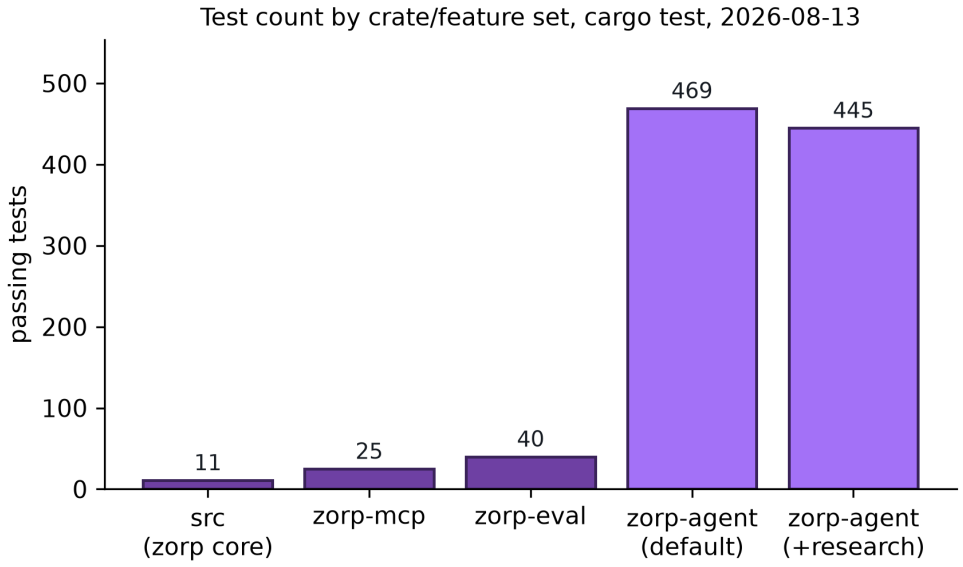

Figure 3: Passing test counts by crate and feature set, from an actual `cargo test` run on 2026-08-13, not aspirational figures.

Table 1: Test counts as of 2026-08-13. *The 469 figure is the sum across all default-workspace test binaries excluding `zorp-track`; it is not directly comparable to the 445 research-feature figure, which is a single `cargo test -p zorp-agent --features research` invocation and includes different integration-test binaries gated on that feature.

Crate / feature set	Passing tests
`src` (zorp core)	11
`zorp-mcp`	25
`zorp-eval`	40
`zorp-agent` (default features)	469*
`zorp-agent` (`--features research`)	445

The workspace is roughly 22,500 lines of Rust across the five crates (`zorp-agent` is by far the largest at approximately 16,800 lines, reflecting that it hosts the full agent: tools, sandbox, trust, verification, sessions, MCP integration, and now the four research capabilities). The repository has had 97 commits since its first commit on 2026-08-08, meaning the entire base harness fork, the `zorp-track` foundation, and all four capabilities described in this paper were built in under a week – a pace consistent with pre-alpha status and a reason to read every claim in this paper as a design and status report, not a mature-system retrospective.

Discussion

What is genuinely built. Every claim in Sections 3 through 5 is backed by code and tests that exist in the repository as of this writing, not a roadmap. The tamper-evidence mechanism, the checkpoint pattern, and the four capabilities' tool-gating logic are all covered by tests that exercise the actual failure modes (a corrupted hash, a non-interactive checkpoint with no `AutoApprove`, a missing MCP tool), not just the happy path.

What is explicitly not claimed. This paper does not report a comparative evaluation against AI-Scientist-v2, Catalyst, or any other system, and it does not report outcome quality for validate's feasibility scores, investigate's evidence, or co-write's drafts on any real investigation. A test suite passing demonstrates that the mechanism behaves as designed; it says nothing about whether the mechanism produces good investigations. That comparison needs a shared task set and a way to judge output quality that does not just reward confident-sounding prose, and building that fairly is future work, not something to gesture at with fabricated numbers.

Where the design constrains itself on purpose. A few decisions in this design trade capability for restraint deliberately: no hard experiment budget (unlike Catalyst's 150-line/10-minute cap), because a budget tuned for one system's narrow validation experiments would be arbitrary for zorp's broader scope; only one track actively worked at a time per session, with true concurrent execution across tracks left for later; and no Hypermemory integration yet, so memory stays local to each track's own stores. None of these are technical limitations so much as scope discipline – each is a real feature deferred rather than quietly dropped.

Future Work

The nearest-term addition to this paper itself, rather than to zorp, is the comparative evaluation described above: a shared task set run through zorp's full loop and through at least AI-Scientist-v2, judged on both process (is the evidence trail complete and tamper-evident) and outcome (is the resulting artifact good), reported honestly including where zorp loses. Beyond that, concurrent multi-track execution, Hypermemory integration for cross-track memory, and LanceDB's actual producers and consumers (literature embeddings for validate, figures for co-write, venue-scope embeddings for deliver) are the next pieces of the foundation to build out, each already scoped in the corresponding design spec but not yet implemented.

Conclusion

zorp is a harness for evidence-based investigation built around a specific bet: that decomposing the problem into four standalone capabilities, each usable alone and chained only through explicit human checkpoints, serves how research and technical decisions actually get made better than one autonomous pipeline that produces a finished document and asks a human to review it afterward. The bet is not yet validated empirically, and this paper says so plainly. What it reports instead is a working system: a tamper-evident record of what was tried, a checkpoint mechanism with no silent defaults, and four capabilities, each backed by real tests, that a person can pick up individually without committing to the whole loop.

Availability

zorp is open source under the MIT license. The repository, including this paper's Markdown source and the scripts used to generate its figures from real repository state, is at `github.com/aviskaar/zorp`.